

栈、队列与受限访问

减少端点自由度，怎样生成可推理的时间秩序

408 · 数据结构 Topic DS-02

母问题：为什么“少允许一些操作”反而能表达过程？

一般线性表允许在许多位置访问和修改；栈、队列与双端队列主动收紧这个自由度。它们的生成链是

Access Restriction → Temporal Order → State Transition

→ Invariant → Application Contract.

这里的箭头依次表示因果生成和检查顺序：先规定哪些端点可进、可出，才能推出元素被取出的时间秩序；每次操作再更新状态字段，并保持空、满、首尾指针和元素次序不变量。应用只依赖这份契约，不应依赖底层是数组还是链表。

本册定位与 Owner 边界

- **Owns**：栈、队列、循环队列、链队列、双端队列的 ADT 契约、物理状态、操作生命周期、空满判据与复杂度。
- **Uses**：DS01 的顺序/链接表示；Atlas 的对象—操作—不变量—成本语言。
- **Outputs**：向 DS-B01 输出可替换的 Frontier 容器契约；向 DS04/DS07 提供遍历所需的取出顺序。
- **Stops**：递归语义、表达式文法、树/图遍历正确性和单调结构的淘汰规则不由本册拥有；这里只说明它们为何能够调用受限访问容器。

目录

1	从线性表到受限访问 ADT	3
2	栈：顶端是唯一活动边界	3
3	队列：两端分工生成 FIFO	3
3.1	为什么顺序队列需要循环化	4
3.2	空与满必须增加一份信息	4
4	双端队列：两端自由，但状态仍受约束	4
5	操作序列、共享空间与表达式：受限访问的题面变体	4
5.1	合法出栈序列不是“看起来像后进先出”	4
5.2	共享栈：两个增长方向共享一个容量不变量	5
5.3	中缀、后缀与表达式求值：运算符栈维护的是未决语义	5
5.4	递归调用栈：每一帧保存尚未完成的未来	5
5.5	括号匹配与单调结构的边界	6

5.6	顺序栈的 top 约定：先画空栈，再写代码	6
5.7	合法出栈序列：模拟与禁用模式	6
5.8	双端队列的三种权限与输出序列	6
5.9	循环队列三种消歧协议与元素个数	7
5.10	表达式的完整栈模型：优先级、结合性与多栈	7
5.11	递归栈与显式栈的等价条件	7
6	代码把访问契约落实为状态机	7
6.1	顺序栈：一份 size 同时分隔有效区与空闲区	7
6.2	循环队列：每次操作只改变一个端点	8
6.3	链队列：删除唯一节点时修复两个端点	9
6.4	双端队列：前端与后端的顺序不对称	10
6.5	边界测试：逼状态跨过零、满与数组末端	11
7	复杂度必须绑定表示与输入契约	12
8	应用只调用时间秩序，不接管容器机制	12
9	概念边界：相似端点名不能替代定义	13
10	做题控制协议与压缩复原	13
11	全科坐标与来源处理	14

1 从线性表到受限访问 ADT

若线性表的状态是一个序列 $S = (a_1, \dots, a_n)$ ，那么“栈”和“队列”的区别不在元素类型，而在允许观察和删除哪一端：

ADT	允许进入	允许离开/观察	推出的次序	典型过程含义
栈 Stack	顶端	顶端	LIFO，后进先出	最近未完成的上下文
队列 Queue	队尾	队头	FIFO，先进先出	最早等待的任务
双端队列 Deque	两端	两端	由调用者选择	可同时模拟栈和队列

限制不是性能结论。栈的 push/pop 可以由顺序表尾部实现，也可以由链表头部实现；只要返回序列一致，二者兑现的是同一 ADT。表示改变的是容量、分配、局部性和维护成本，不是 LIFO/FIFO 的抽象结果。

完整生成例：三项任务按栈和队列退出

依次加入 A, B, C 。若只允许从加入端退出，则状态轨迹为

$$[] \rightarrow [A] \rightarrow [A, B] \rightarrow [A, B, C] \rightarrow [A, B] \rightarrow [A] \rightarrow [],$$

退出序列是 C, B, A 。若一端加入、另一端退出，则退出序列是 A, B, C 。同一组元素没有改变，访问限制却确定了时间秩序；这正是应用选择容器时真正购买的语义。

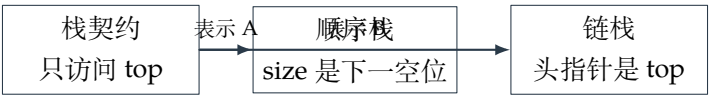
2 栈：顶端是唯一活动边界

把栈状态写作 $(data, size, capacity)$ 或 $(top, size)$ 。顺序栈的合法状态是

$$0 \leq size \leq capacity, \quad top = data[size - 1] \quad (size > 0).$$

执行 push(x) 时先判满，再写入 $data[size]$ 并递增长度；执行 pop() 时先判空，再递减长度并读取原顶元素。**先减后读**与**先读后减**都可正确实现，但下标和返回值必须指向同一个旧顶元素。

链栈把头节点作为栈顶：压栈令新节点的后继指向旧顶，再更新顶指针；出栈先保存旧顶，再跨过它并释放。若把尾节点当栈顶而底层又是单链表，出栈还要寻找前驱，便丢掉了预期的 $O(1)$ 局部更新。



3 队列：两端分工生成 FIFO

队列允许在队尾 enqueue、在队头 dequeue。链队列通常保存 $(front, rear, size)$ ：入队接到 $rear$ 后，出队从 $front$ 删除。最危险的不是一般节点，而是最后一个节点出队：新 $front$ 为空时， $rear$ 也必须复位为空，否则“空队列”会残留悬空尾指针。

3.1 为什么顺序队列需要循环化

若数组队列只让 $front, rear$ 单调右移, 前部空槽无法复用, 会出现“数组还有空间但队尾到界”的假溢出。循环队列把下标放在模 M 的环上:

$$\text{next}(i) = (i + 1) \bmod M.$$

这不是为了背取模公式, 而是把已出队的槽重新纳入可用状态空间。

3.2 空与满必须增加一份信息

只保存 $front, rear$ 且二者都指向下一操作位置时, $front = rear$ 既可能表示空, 也可能表示绕一圈后的满。实现必须选择一种消歧协议:

协议	空判据	满判据	代价/适用点
牺牲一个槽	$front = rear$	$\text{next}(rear) = front$	可用容量为 $M - 1$
长度计数	$size = 0$	$size = M$	多维护一个计数字段
标志/最近操作	$front = rear$ 且标志为空	$front = rear$ 且标志为满	分支语义更复杂

本册伴随实现选择长度计数, 因此合法状态还要求

$$0 \leq size \leq M, \quad rear \equiv front + size \pmod{M}.$$

不能把该实现的 $size = M$ 满判据与“牺牲一个槽”实现的 $\text{next}(rear) = front$ 混用。

4 双端队列：两端自由，但状态仍受约束

双端队列允许 $\text{push_front}/\text{push_back}$ 与 $\text{pop_front}/\text{pop_back}$ 。栈和普通队列都可由它限制操作集合得到:

$$\text{Deque} \xrightarrow{\text{只同端进出}} \text{Stack}, \quad \text{Deque} \xrightarrow{\text{一端进、另一端出}} \text{Queue}.$$

这说明 Deque 的 ADT 能力更强, 却不等于所有应用都应使用它。更宽的接口允许更多非法调用组合; 若应用只需要 FIFO, 直接暴露 Queue 契约更能保护时间秩序。

循环双端队列仍维护 $rear \equiv front + size \pmod{M}$ 。前端插入要先令 $front$ 向前回绕再写入; 后端删除要先令 $rear$ 向前回绕再读取。顺序写反会读到“下一空位”, 产生典型的 off-by-one 错误。

5 操作序列、共享空间与表达式：受限访问的题面变体

5.1 合法出栈序列不是“看起来像后进先出”

给定输入序列 $1, 2, \dots, n$, 每一步可以继续入栈, 也可以在栈非空时出栈。判定一个候选输出序列时, 维护“已经读入但尚未输出”的栈:

1. 若栈顶等于当前候选输出, 立即出栈并前进输出指针;
2. 否则若输入尚未耗尽, 继续把下一个输入压栈;
3. 若输入耗尽且栈顶仍不是候选输出, 序列非法。

因此合法性是一个状态可达性问题，不是只检查“有没有重复”。例如 1,2,3 可以输出 2,3,1（先压 1,2，出 2，再压 3，出 3,1），但不能输出 3,1,2，因为输出 3 时 1,2 都在它下面，随后不可能先取出 1。计数题还必须区分“输入次序固定的栈排列”与“任意元素可直接访问的排列”，后者不再是栈问题。

双端队列题要先声明输入限制：输入只能从一端进入、输出只能从一端离开，还是两端都可用。不同限制会产生不同可行序列；把“输入受限 Deque”和普通 Queue 混成同一张表，会丢掉题目真正考察的端点约束。

5.2 共享栈：两个增长方向共享一个容量不变量

在同一数组中让两个栈相向增长，典型状态为 top_1 指向左栈顶、 top_2 指向右栈顶。若两端均把“下一空位”作为边界，栈满条件是

$$top_1 + 1 = top_2.$$

这里的“满”不是某一个栈达到数组末端，而是两条有效区域即将相撞。左栈压入会增大 top_1 ，右栈压入会减小 top_2 ；出栈只改变对应一端。共享栈的收益是两个栈可以互相借用暂时空出的槽位，代价是每个操作都必须检查全局相撞条件。若题目采用“top 指向栈顶元素”而不是“下一空位”，公式会整体平移一格，必须先画空栈状态再推导，不能直接背上式。

5.3 中缀、后缀与表达式求值：运算符栈维护的是未决语义

表达式题的核心不是把字符串换一种写法，而是把“尚未决定结合次序的运算符”保存到栈中。中缀转后缀时维护两个区域：输出序列已经确定的 token，以及运算符栈中等待比较优先级的 token。

1. 扫到操作数，直接输出；
2. 扫到左括号，压栈并把它视为新的优先级边界；
3. 扫到右括号，不断弹出直到左括号，左右括号本身不输出；
4. 扫到运算符 op ，若栈顶运算符优先级不低于 op 且不跨越左括号，就弹出栈顶，再压入 op ；
5. 扫描结束后把栈中剩余运算符依次弹出。

“大于等于”而非“大于”决定同优先级运算符的左结合；若题目允许右结合的幂运算，比较规则必须单独写出。表达式求值时，操作数栈每遇到二元运算符弹出右操作数 b 、再弹出左操作数 a ，计算 $a \text{ op } b$ 后压回；弹出顺序不能交换。除法、负号、括号和非法 token 都是边界分支，不应被“栈算法”四个字掩盖。

5.4 递归调用栈：每一帧保存尚未完成的未来

递归不是“函数自己调用自己”的语法现象，而是把每次调用的局部状态压入系统栈。每个递归函数至少要回答三个问题：

1. 参数表示的子问题是什么，规模怎样变小；
2. 终止条件是否覆盖空结构、叶节点和目标已命中；
3. 子调用返回后，当前帧还要执行什么合并、回溯或访问动作。

对树遍历，访问语句放在递归调用前、两次调用之间或两次调用之后，分别生成前序、中序和后序；这正是“返回时机”与栈帧恢复顺序的关系。把递归改写为显式栈时，不能只保存节点，还要保存“下一步要执行哪个子调用”的程序计数状态，否则只能得到某一种遍历而不能等价模拟原递归。

递归空间复杂度应计入最大同时存在的调用帧数。递归深度为 h 的树遍历使用 $O(h)$ 栈空间；若树退化成链， $h = n$ ，不能仅凭“每个节点访问一次”写成 $O(1)$ 额外空间。尾递归是否能由编译器消除是实现扩展，不能替换 408 题目要求的抽象调用栈分析。

5.5 括号匹配与单调结构的边界

括号匹配是最小的栈应用：遇到左括号压入类型，遇到右括号时必须检查栈非空且栈顶类型匹配，扫描结束后栈必须为空。三类失败分别是“右括号无对应左括号”“类型不匹配”“仍有未闭合左括号”。

单调栈、单调队列是在普通受限访问之外增加了候选淘汰不变量：新元素到来时，所有不可能再成为答案的旧元素被从端点弹出。它们仍使用 DS02 的端点操作，但“为什么可以淘汰”属于具体问题的支配关系/窗口约束，不属于普通栈或队列定义。规则进入考试控制或对应算法题，不在本册扩成第四种容器。

5.6 顺序栈的 top 约定：先画空栈，再写代码

顺序栈常见两种等价口径： top 指向当前栈顶元素，空栈为 $top = -1$ ；或 top 指向下一空位，空栈为 $top = 0$ 。前者入栈是“先加后写”、出栈是“先读后减”；后者入栈是“先写后加”、出栈是“先减后读”。两种口径都能得到同一 LIFO 结果，错误只发生在初始化、判空、判满和读写顺序混用时。动态顺序栈还要明确 $base$ 指向连续存储区、 $top - base$ 是元素数量；链栈则没有固定上界，失败条件是分配失败而不是数组上溢。

5.7 合法出栈序列：模拟与禁用模式

输入顺序固定为 $1, 2, \dots, n$ 时，出栈过程可看作“从输入流继续压入”与“从栈顶输出”两种动作的交错。最可靠的判定算法是维护输入指针、辅助栈和输出指针：栈顶等于目标就弹出；否则继续压入尚未读取的输入；输入耗尽且栈顶仍不匹配即失败。任何候选输出都必须同时满足每个元素只出现一次、输入次序不被跳过以及每次输出确实在栈顶。

还有一个可用于复核的禁用模式：若输出序列中存在 $a < b < c$ ，且 a 在 b, c 之前而 c 在 b 之前（相对次序呈 a, c, b ），则它不可能由固定输入栈产生； a 压入后 b 在它上面，若先取出更晚的 c ， b 会被挡住而不能随后取出。计数时，固定输入顺序的合法出栈序列数是 Catalan 数

$$C_n = \frac{1}{n+1} \binom{2n}{n},$$

它等价于“入栈/出栈前缀中任意时刻出栈数不超过入栈数”的 Dyck 路径计数；不要把它与任意排列的 $n!$ 混淆。括号匹配、二叉树形状和合法栈操作序列中出现同一递推，是因为它们都在维护一个不能先耗尽的未决边界。

5.8 双端队列的三种权限与输出序列

题面中的“受限双端队列”必须先标出输入端和输出端：

类型	允许操作	判题重点
输入受限	只能从一端输入，两端可输出	每个新元素进入固定端，输出时可选两端
输出受限	两端可输入，只能从一端输出	需模拟两端输入如何排队到唯一出口
完全双端	两端均可输入/输出	能力最强，但若题面未限制则不能擅自增加操作

输入受限时，候选输出序列可用首尾剥离规则快速复核：读入下一个元素前，目标值若不是当前两端之一就必须继续读；若两端均不匹配且输入耗尽则失败。输出受限时没有同样的一眼规则，按“从左端/右端输入”逐步模拟更稳妥。栈和队列只是双端队列的两个操作子集，不能因为某序列能由 Deque 产生就断言它能由 Stack 或 Queue 产生。

5.9 循环队列三种消歧协议与元素个数

同一数组环可以有套合法设计，必须整套使用：

1. 牺牲一个槽： $front = rear$ 表示空， $next(rear) = front$ 表示满，最多存 $M - 1$ 个元素；
2. 保存 size：空为 $size = 0$ ，满为 $size = M$ ，且 $size = (rear - front + M) \bmod M$ （若 $rear$ 指下一空位）；
3. 保存 tag：入队/出队后端点相遇时分别把标志置为“满/空”，容量可用满 M 个槽，但状态转移多一条标志维护。

若 $rear$ 指向队尾元素而不是下一空位，元素个数公式和入/出队顺序会整体平移；正确做法是先画“空队列”和“恰好一个元素”的指针图，再推公式。所谓“假溢出”只属于非循环顺序队列前端空槽未复用的实现缺陷，不属于逻辑队列容量本身。

5.10 表达式的完整栈模型：优先级、结合性与多栈

中缀转后缀用运算符栈时，栈内优先级与栈外优先级需要区分：左括号入栈后形成新的边界，右括号触发弹栈直到左括号；同优先级左结合运算符遇到新运算符时应先弹出，右结合运算符（若题目允许）则保留栈顶。扫描结束必须把剩余运算符弹出，否则后缀表达式缺少尾部运算。

后缀求值只需操作数栈：遇操作数压入，遇二元运算符先弹右操作数 b 、再弹左操作数 a ，计算 $a \text{ op } b$ 。中缀直接求值则维护“操作数栈 + 运算符栈”两个状态；新运算符优先级不高于栈顶时先完成栈顶运算。前缀求值从右向左扫描，弹出顺序仍是先得到左/右操作数后按运算符语义组合。表达式树把三种形式统一为前序/中序/后序遍历；中缀输出必须按子树优先级补括号，不能只输出一一次中序遍历就声称无歧义。

5.11 递归栈与显式栈的等价条件

一次函数调用要在系统栈帧中保存返回地址、参数、局部变量和下一条待执行位置；递归返回时恢复该帧，所以“访问语句放在递归调用前/中/后”分别对应前序/中序/后序。把递归改成显式栈时，若原函数在返回后还要处理右子树，栈项必须额外保存节点和阶段标记（例如“左子树已完成，下一步处理右子树”），只压入节点指针会丢失控制状态。递归时间看调用树节点总数，递归空间看同时存在的根到当前节点路径；两者可以分别是 $O(n)$ 和 $O(h)$ 。

6 代码把访问契约落实为状态机

完整 C++17 实现位于 `code/ds02_restricted_access.hpp`，测试位于 `code/ds02_restricted_access_test.cpp`。以下代码块直接抽取真实源文件，正文解释与可执行实现共享同一来源。

6.1 顺序栈：一份 size 同时分隔有效区与空闲区

```
1  bool empty() const { return size_ == 0; }
2  bool full()  const { return size_ == capacity_; }
```

```

3  std::size_t size() const { return size_; }
4
5  void push(int value) {
6      if (full()) {
7          throw std::overflow_error("stack overflow");
8      }
9      data_[size_++] = value;
10 }
11
12 int pop() {
13     if (empty()) {
14         throw std::underflow_error("stack underflow");
15     }
16     return data_[--size_];
17 }
18
19 int top() const {
20     if (empty()) {
21         throw std::underflow_error("stack is empty");
22     }
23     return data_[size_ - 1];
24 }
25
26 bool invariant() const {
27     return size_ <= capacity_ && (capacity_ == 0 || data_ != nullptr);
28 }

```

代码清单 1: 顺序栈的空满、压栈、出栈与不变量

代码把有效区定义为 $[0, size)$ ，因此压栈写 `data_[size_]` 后递增，出栈递减后读取。满/空检查在访问数组之前执行，容量为 0 时也只会进入上溢分支，不会解引用空存储。

6.2 循环队列：每次操作只改变一个端点

```

1  bool empty() const { return size_ == 0; }
2  bool full() const { return size_ == capacity_; }
3  std::size_t size() const { return size_; }
4
5  void enqueue(int value) {
6      if (full()) {
7          throw std::overflow_error("queue overflow");
8      }
9      data_[rear_] = value;
10     rear_ = next(rear_);
11     ++size_;
12 }
13
14 int dequeue() {
15     if (empty()) {
16         throw std::underflow_error("queue underflow");
17     }
18     const int value = data_[front_];

```



```

19     front_ = next(front_);
20     --size_;
21     return value;
22 }
23
24 int front() const {
25     if (empty()) {
26         throw std::underflow_error("queue is empty");
27     }
28     return data_[front_];
29 }
30
31 bool invariant() const {
32     if (size_ > capacity_) {
33         return false;
34     }
35     if (capacity_ == 0) {
36         return data_ == nullptr && front_ == 0 && rear_ == 0 && size_ == 0;
37     }
38     return data_ != nullptr && front_ < capacity_ && rear_ < capacity_ &&
39         rear_ == (front_ + size_) % capacity_;
40 }

```

代码清单 2: 循环队列的入队、出队与环形不变量

入队只写 *rear* 所指的下一空位并推进 *rear*；出队只读 *front* 所指的首元素并推进 *front*。两者分别增减 *size*，共同维护 $rear = (front + size) \bmod capacity$ 。next 对零容量单独返回 0，但正常操作在此前已由空满条件拦截。

6.3 链队列：删除唯一节点时修复两个端点

```

1  void enqueue(int value) {
2      Node* node = new Node{value, nullptr};
3      if (rear_ == nullptr) {
4          front_ = rear_ = node;
5      } else {
6          rear_->next = node;
7          rear_ = node;
8      }
9      ++size_;
10 }
11
12 int dequeue() {
13     if (empty()) {
14         throw std::underflow_error("queue underflow");
15     }
16     Node* removed = front_;
17     const int value = removed->value;
18     front_ = removed->next;
19     if (front_ == nullptr) {
20         rear_ = nullptr;

```

```

21     }
22     delete removed;
23     --size_;
24     return value;
25 }
26
27 int front() const {
28     if (empty()) {
29         throw std::underflow_error("queue is empty");
30     }
31     return front_->value;
32 }
33
34 bool invariant() const {
35     if ((size_ == 0) != (front_ == nullptr && rear_ == nullptr)) {
36         return false;
37     }
38     std::size_t count = 0;
39     Node* last = nullptr;
40     for (Node* current = front_; current != nullptr; current = current->next) {
41         last = current;
42         ++count;
43     }
44     return count == size_ && last == rear_ &&
45         (rear_ == nullptr || rear_>next == nullptr);

```

代码清单 3: 链队列的首尾迁移与不变量

一般出队只移动 `front_`；当旧队列只有一个节点时，移动后 `front_` 为空，代码同步把 `rear_` 置空。`invariant()` 再检查空状态等价、可达节点数、尾节点身份和尾后继为空。

6.4 双端队列：前端与后端的顺序不对称

```

1  void push_front(int value) {
2      require_space();
3      front_ = previous(front_);
4      data_[front_] = value;
5      ++size_;
6  }
7
8  void push_back(int value) {
9      require_space();
10     data_[rear_] = value;
11     rear_ = next(rear_);
12     ++size_;
13 }
14
15 int pop_front() {
16     require_element();
17     const int value = data_[front_];
18     front_ = next(front_);
19     --size_;

```

```

20     return value;
21 }
22
23 int pop_back() {
24     require_element();
25     rear_ = previous(rear_);
26     const int value = data_[rear_];
27     --size_;
28     return value;
29 }
30
31 int front() const {
32     require_element();
33     return data_[front_];
34 }
35
36 int back() const {
37     require_element();
38     return data_[previous(rear_)];
39 }
40
41 bool invariant() const {
42     if (size_ > capacity_) {
43         return false;
44     }
45     if (capacity_ == 0) {
46         return data_ == nullptr && size_ == 0 && front_ == 0 && rear_ == 0;
47     }
48     return data_ != nullptr && front_ < capacity_ && rear_ < capacity_ &&
49         rear_ == (front_ + size_) % capacity_;

```

代码清单 4: 循环双端队列的四种端点更新

front_ 始终指向首元素, rear_ 始终指向尾后下一空位。因此前插是“先退再写”, 后插是“先写再进”; 前删是“先读再进”, 后删是“先退再读”。四个操作不是可随意对称替换的口诀, 而是由端点语义生成。

6.5 边界测试: 逼状态跨过零、满与数组末端

```

1
2 void test_circular_queue_wraparound() {
3     CircularQueue zero_capacity(0);
4     assert(zero_capacity.empty() && zero_capacity.full());
5     assert_throws<std::overflow_error>([&] { zero_capacity.enqueue(1); });
6     assert(zero_capacity.invariant());
7
8     CircularQueue queue(3);
9     queue.enqueue(10);
10    queue.enqueue(20);
11    queue.enqueue(30);
12    assert_throws<std::overflow_error>([&] { queue.enqueue(40); });

```

```
13  assert(queue.dequeue() == 10);
14  queue.enqueue(40);
15  assert(queue.front() == 20);
16  assert(queue.dequeue() == 20);
17  assert(queue.dequeue() == 30);
18  assert(queue.dequeue() == 40);
19  assert(queue.empty() && queue.invariant());
20 }
21
22 void test_linked_queue_singleton() {
23     LinkedQueue queue;
24     assert_throws<std::underflow_error>([&] { queue.dequeue(); });
}
```

代码清单 5: 循环队列满状态、回绕与 FIFO 断言

测试先填满容量为 3 的队列，验证第四次入队上溢；再出队一个元素并入队 40，迫使 *rear* 回绕，最后检查退出仍为 10,20,30,40 的 FIFO 次序。测试集还覆盖零容量、空栈/空队列下溢、链队列删至空后复用，以及双端队列两端交错操作。

```
1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
2     code/ds02_restricted_access_test.cpp -o /tmp/ds02_test
3 /tmp/ds02_test
```

7 复杂度必须绑定表示与输入契约

操作	顺序表示	链式表示	成本来源	边界条件
栈 push/pop	$O(1)$	$O(1)$	尾下标或头指针局部更新	定容上溢/动态分配
队列 en-queue/dequeue	$O(1)$	$O(1)$	环形端点或首尾指针	正确空满判据
访问 top/front/back	$O(1)$	$O(1)$	直接端点	非空
检查结构不变量	$O(1)$	$O(n)$	字段关系 vs 全链遍历	属于测试成本
存储 n 个元素	$O(M)$	$O(n)$	定容槽位 vs 每节点指针	$n \leq M$

若顺序栈采用动态扩容，单次扩容为 $\Theta(n)$ ，连续压栈可讨论摊还 $O(1)$ ；本册实现采用固定容量，因而满时显式抛出上溢，不能悄悄套用动态数组结论。

8 应用只调用时间秩序，不接管容器机制

- **递归/括号/表达式**：需要恢复最近尚未完成的上下文，因此调用 LIFO；文法和递归语义由相应专题解释。
- **层序遍历/BFS**：需要先展开较早发现的 *frontier*，因此调用 FIFO；“首次发现得到无权最短边数”由 DS07 的图模型证明。
- **DFS**：显式栈或调用栈都提供 LIFO *frontier*；访问标记、完成时序和图性质仍由 DS07 Own。

- **单调栈/单调队列**：它们在端点限制之外又增加“发现更优候选时淘汰旧候选”的不变量；该淘汰规则不是普通栈/队列定义。

Frontier Bridge 的接口语言

调用者只需回答：新发现状态怎样加入？下一状态按什么策略取出？何时算已处理？普通 Stack/Queue 提供前两项的 LIFO/FIFO 策略；DS-B01 负责把这份接口桥接到树和图，DS04/DS07 再提供各自的正确性不变量。

9 概念边界：相似端点名不能替代定义

概念 A	≠ 概念 B	真正判据	混淆后果
ADT 次序	≠ 物理表示	返回序列 vs 数组/节点布局	把实现细节当定义
rear 元素	≠ rear 下标	本册 rear 指向尾后空位	读取错误槽位
队空	≠ front=rear	该等式必须结合消歧协议	把满队误判为空
上溢	≠ 假溢出	真无槽位 vs 未复用前部槽位	错判容量状态
Deque 能力	≠ Queue 契约	能两端操作 vs 只允许 FIFO	应用可破坏时间秩序
容器不变量	≠ 遍历正确性	结构合法 vs 算法输出正确	Owner 越界

最常见的错误推理

“循环队列就是下标取模”漏掉空满消歧；“front/rear 的公式可以直接套”漏掉二者究竟指向元素还是空位；“链式队列不会满”忽略分配失败和资源约束；“BFS 用了队列，所以最短路由队列证明”混淆容器顺序与图算法不变量。

10 做题控制协议与压缩复原

遇到受限访问结构，按以下顺序建模：

1. 写操作集合：哪一端允许进入、离开和观察；
2. 写清端点字段语义，尤其 front/rear 指元素还是空位；
3. 选择空满消歧协议，不混用其他协议的公式；
4. 画一次普通状态和一次跨边界状态，逐步更新字段；
5. 验证抽象退出次序和物理不变量；
6. 最后把复杂度绑定到表示、容量策略和输入前提。

一页压缩

忘掉公式后，从五问复原：**允许动哪一端？这生成什么时间秩序？端点字段各指什么？空与满靠哪份额外信息区分？一次操作后哪些字段必须一起修复？**

栈用同端进出生成 LIFO；队列用异端进出生成 FIFO；双端队列提供更宽能力。循环化只解决槽位复用，空满消歧才决定状态是否可判；应用正确性必须继续交给调用该容器的 Topic。

11 全科坐标与来源处理

Map Coordinate: Restriction / Order / State / Application

DS02 接收 DS01 的顺序与链接表示，把“访问自由度”压缩为 LIFO/FIFO/Deque 契约；再经 DS-B01 把取出策略交给 DS04 的树遍历和 DS07 的图遍历。数据结构 Atlas 中“表示—操作—不变量—成本”在本册具体化为端点语义、状态字段与边界分支。

本册吸收归档总册中“减少自由度换取过程语义”“循环队列关键在空满规则而非取模口诀”以及 Frontier 统一视角。旧材料中 BFS/DFS、括号匹配、表达式和单调结构仅作为 Use 证据：其专属机制不提升为 DS02 的稳定定义。伴随实现承担代码级机制证据；测试承担边界与不变量证据。